后训练学习计划 • 第 11 周（11/16-11/22）

本周定位：**Phase 3 / M4 第二周——对齐阶段**。把第 8 周的 DPO 管线与第 9 周的 GRPO 管线移植到明日记忆 Tutor 的数据上，产出 `SFT → SFT+DPO → SFT+DPO+GRPO` 三档模型和一份对齐前后效果对比表。 这是 16 周里技术密度最高、最容易翻车的一周：reward 不涨、JSON 格式崩、通用能力退化，三件事至少会撞上一件。本周每个环节都配了诊断路径与降级方案。
用法说明：每天先看「今日总览」，再从任务 1 逐条做；每个任务都有 **完成判据**，判据不满足不要进入下一个任务。**加粗红字 = 硬性要求**，灰色小字块 = 注释（可跳过，不影响主线）。

一、本周总览

天	主题	核心硬产出	预计
Day 1 周一	移植体检 + tutor 偏好对构造	`pref_tutor_*.jsonl` 训练集 ≥1200 对 + 100 条人工复核一致率	6h
Day 2 周二	DPO 上卡（tutor 数据）	`dpo-merged` 权重 + held-out 偏好对 acc 达标	6.5h
Day 3 周三	可验证奖励三件套（纯本地，不上卡）	`rewards.py` + 15 个单测全绿 + 奖励分布体检表	6h
Day 4 周四	GRPO 上卡（本周最大风险）	reward / KL / 输出长度三条曲线 + 收敛或降级的书面判定	7h
Day 5 周五	对齐前后三档对比表 + 人工盲评	`RESULTS.md` 对比表 + 40 题盲评记录	6h
Day 6 周六	M4 系列博客第 2 篇 + 失败案例归档	博客已发布 + `FAILURES.md` ≥6 条	6h
Day 7 周日	补漏 + 闭卷复测 + 总验收 + 周 12 预习	验收 ≥6/7 项通过	4.5h

注：**本周卡时上限 14h，预估花费 ≈¥28**（4090 按量 ¥1.5-2.5/时）。分配：Day 1 冒烟 0.5h、Day 2 训练 4h、Day 4 训练 6h、Day 5 生成 2h、Day 6 补图 0.5h，**剩 1h 作排队与重跑缓冲**（Day 3 若需要开卡采样 0.5h，就从这 1h 里出）。**超了怎么办**：先砍 Day 5 的上卡生成（降级成本地 0.5B 出图，数字不好看但表结构不变），再砍 Day 2 的第二遍重跑；**Day 4 的 6h 不许砍**——GRPO 跑不满一个 epoch 就没有判定收敛的资格。全程 1.5B + LoRA，不许擅自换 7B：4090 上 7B 的 GRPO（policy + ref + 激活 + rollout）显存不够，这不是调参能救的。

注：**本周最大风险不是难度，是三样可能同时坏**。（1）reward 不涨 → Day 3 的奖励分布体检就是为它准备的，别跳；（2）格式崩（对齐后 JSON 不合法率上升）→ Day 2、Day 4 各有硬判据；（3）能力退化 → Day 5 的退化快测 + 周 12 的正式基准。**砍单顺序（实在做不完按这个砍）**：Day 5 任务 4 退化快测 → Day 6 任务 3 的失败案例配图 → Day 1 任务 4 人工复核从 100 条砍到 50 条（一致率照算）→ **其余不许砍**。

注：一条底线，来自总计划风险预案第 1 条：**GRPO 不收敛也算本周完成**。判定不收敛就走降级路线——交付 `SFT+DPO` 两版，并在 `RESULTS.md` 与博客里写清"为什么不收敛、定位到哪一步、为什么退回 DPO"。一份带失败曲线的记录，比一份假装成功的表值钱得多。

二、开始前：环境 / 素材 / 上卡准备（约 35 分钟，前一晚做完）

1. **本地目录与依赖**（bash 里一律用正斜杠，反斜杠会被吞）：

```
mkdir -p E:/study/posttrain/week11/{day1,day2,day3,day4,day5,day6}
cd E:/study/posttrain/llm-posttraining-lab && git pull
source E:/study/posttrain/.venv/Scripts/activate
pip config set global.index-url https://pypi.tuna.tsinghua.edu.cn/simple
pip install jsonschema pytest pandas -q
```

2. **周 10 产物核对清单**（缺一样先在 `gaps.md` 立条目，当天优先补，不要硬开工）：
   - `m4-capstone/sft/`：周 10 的 SFT checkpoint 与 merge 后的 `tutor-sft-1.5b`（本地留 config/tokenizer，权重在卡上 `/root/autodl-tmp/m4/`）。
   - `m4-capstone/ab-protocol.md`：A/B 协议是否已**冻结**——同一组 30 个真实查词场景 + 记录字段（延迟 / 成本 / 主观分）。本周不许动字段，动了周 12 的 A/B 就对不齐。
   - 周 7 数据集 `english-tutor-instruct`：确认留有**未参与 SFT 的 prompt-only 切分 ≥800 条**。没有就 Day 1 任务 3 现场用周 7 的过滤脚本重跑一遍、只取 prompt。
   - 周 8 的 `m2-sft/dpo/run_dpo.py`、周 9 的 `m3-grpo/`：本周是**移植**，不是从零重写。

> 注：**从上周 gaps 清这三条**（Day 1 任务 1 正式动手，先扫一眼）：(a) 周 10 遗留的"LoRA merge 之后 chat template 与特殊 token 是否还在"——这条不对齐，DPO 出来格式必崩；(b) "A/B 的 30 个场景没落成 jsonl"；(c) 周 9 遗留的"reward 函数没有单测"——本周它要直接接进 GRPO，没单测等于给训练埋雷。

3. **AutoDL 启动脚本模板**（前一晚写好，开卡当天直接调用，不要临场手敲）：

```
#!/usr/bin/env bash
# start_w11.sh —— 放 /root/autodl-tmp/m4/ 下，用法: bash start_w11.sh run_dpo_tutor.py
set -uo pipefail
source /etc/network_turbo || true          # 学术加速，对 github/hf/pypi 有效，失败就跳过
export HF_ENDPOINT=https://hf-mirror.com
export WANDB_MODE=online                   # 连不上改 offline，回头 wandb sync 补
mkdir -p /root/autodl-tmp/m4/logs
cd /root/autodl-tmp/m4
python "$1" 2>&1 | tee "logs/$1_$(date +%m%d_%H%M).log"
```

> 注：`/root` 下的内容可能随镜像重置丢，**权重与 jsonl 一律落 `/root/autodl-tmp`（持久盘）**，并且每天收工 `git push` 一次脚本与数据。关机 ≠ 销毁：离开终端前在控制台点**关机**、确认状态是"已关机"；任何时候不要点销毁——本周你手上有 3 份权重，销毁了就报仇无门。

Day 1 • 周一：移植体检 + tutor 偏好对构造

今日总览（6h）：今天不跑大训练，只做两件事：把周 8/9 的管线在 0.5B + 20 条样本上**空跑到通**（证明移植没坏），然后把 tutor 的偏好对数据造出来。晚上要拿到 `pref_tutor_train.jsonl` / `pref_tutor_test.jsonl`（训练集 **≥1200 对**），并完成 100 条人工复核、算出标注一致率。卡时约 0.5h（任务 2 的冒烟）。

任务 1：素材清点 + merge 后的 template 体检（约 40 分钟）

1. 打开 `E:/study/posttrain/gaps.md`，把"从上周清这三条"抄进今日待办并标状态。
2. 核对「开始前」第 2 步的清单，缺什么当场记进 gaps。
3. 开卡（0.5h 够了），把下面脚本存成 `m4-capstone/align/smoke_template.py` 并跑：

```
# smoke_template.py —— 三查：template / 特殊标记 / 输出能否解析为 JSON
import json
from transformers import AutoTokenizer, AutoModelForCausalLM
P = "/root/autodl-tmp/m4/sft-merged"
tok = AutoTokenizer.from_pretrained(P)
msgs = [{"role": "system", "content": "你是英语老师，只输出 JSON。"},
        {"role": "user", "content": "为单词 serendipity 生成教学内容，目标水平 B1。"}]
text = tok.apply_chat_template(msgs, tokenize=False, add_generation_prompt=True)
print("HAS_CHATML_MARKERS:", ("im_start" in text and "im_end" in text))
print("TEMPLATE_HEAD:", repr(text[:160]))
m = AutoModelForCausalLM.from_pretrained(P, torch_dtype="bfloat16", device_map="cuda:0").eval()
enc = tok(text, return_tensors="pt").to("cuda:0")
out = m.generate(**enc, max_new_tokens=320, do_sample=True, temperature=0.2, top_p=0.9,
                 pad_token_id=tok.pad_token_id or tok.eos_token_id)
gen = tok.decode(out[0][enc.input_ids.shape[1]:], skip_special_tokens=True)
print("GEN_HEAD:", gen[:200])
try:
    keys = sorted(json.loads(gen).keys())
    print("PARSE_OK:", keys)
except Exception as e:
    print("PARSE_FAIL:", type(e).__name__, e)
```

完成判据：打印 `HAS_CHATML_MARKERS: True`；`PARSE_OK` 里四个键齐（`mnemonic` / `wordRoot` / `sceneExamples` / `confusable`）。**若 `PARSE_FAIL`，不要往下走**——先回查是不是周 10 保存时只存了 adapter 没 merge tokenizer，这是移植的头号拦路虎。

> 注：输出契约就是明日记忆服务端在用的那份：`packages/agent/src/nodes/tutor.ts` 里的 `TutorContentSchema`（`mnemonic`、`wordRoot`、`sceneExamples` 恰好 3 条且每条含 `en` / `zh` / `scene`、`confusable`）。**对齐阶段不许改字段**——改了等于换任务，周 12 的线上 A/B 直接对不上冻结协议。

任务 2：管线空跑（0.5B + 20 条，约 60 分钟）

1. 复制周 8 的 DPO 脚本为 `m4-capstone/align/run_dpo_tutor.py`、周 9 的 GRPO 脚本为 `m4-capstone/align/run_grpo_tutor.py`，只改三处：模型路径、数据路径、reward 函数导入路径。**其余一律不动**（保留周 8/9 调好的超参）。
2. 造 20 条假数据 `mini_pref.jsonl`（从周 7 prompt 里取 10 条，每条配一个"完整合法 JSON"当 chosen、一个"截断 + 缺字段"当 rejected）。
3. 空跑（`max_steps=2`、`per_device_train_batch_size=1`、0.5B）：

```
python run_dpo_tutor.py --model Qwen/Qwen2.5-0.5B --data mini_pref.jsonl --max_steps 2
python run_grpo_tutor.py --model Qwen/Qwen2.5-0.5B --prompts mini_prompts.jsonl --max_steps 2 \
    --num_generations 2 --rewards m4.capstone.align.rewards:dummy_reward
```

（参数名以你环境里 `trl.__version__` 对应文档为准；报错提示会告诉你正确字段名，**这项排查限时 20 分钟**，超时就记 gaps 明天再弄，别在 Day 1 烧掉半天。）

完成判据：两个脚本都跑到 `train_loss` 有打印且无 NaN，末行输出 `PIPELINE SMOKE OK`（自己在脚本末尾 `print` 一句）。

> 注：`reward_funcs=[...]` 的写法在 TRL 新版本里是 `"模块路径:函数名"` 或函数对象两种，函数签名在版本间也变过（旧版收 `(prompts, completions)`，新版只收 `(completions, **kwargs)`，数据集列名会作为 kwargs 传进来）。**不要靠记忆写，靠报错写**：先在 Python 里 `import trl; print(trl.__version__)`，把版本记进 `requirements-w11.txt`，本周全程锁死这个环境，不许中途升级。

任务 3：tutor 偏好对构造（约 150 分钟，本周第一个真产出）

1. 取 prompt：从周 7 的 prompt-only 切分里取 **600 个不同单词 × 目标 CEFR 档**（A1/A2/B1/B2/C1 各约 120 条），存 `m4-capstone/align/prompts_600.jsonl`，字段 `{word, target_level, prompt}`。
2. 每个 prompt 用 **周 10 的 SFT 模型**采 4 个输出（`temperature=1.0, top_p=1.0, seed=42, max_new_tokens=512`）→ 2400 条候选。**先本地 0.5B 把脚本跑通，再上卡跑 1.5B**（这是全程通用的省钱顺序）。
3. 给每条候选打分 `score`，打分器 = 规则分（Day 3 的三件套雏形，今天先用简版：JSON 可解析 0/1、`sceneExamples` 数量是否为 3、英文例句是否含目标词）+ 强模型分（DeepSeek，1-10 的主观质量分）。**最终分 = 0.4×规则归一分 + 0.6×强模型分**。
4. 同一 prompt 内取 `max-min` 配对，且要求两者分差 ≥2（强模型分口径），否则丢弃该 prompt。
5. 按 9:1 切 train/test，写 `pref_tutor_train.jsonl` / `pref_tutor_test.jsonl`，字段 `{prompt, chosen, rejected, word, target_level, scores}`。

完成判据：`pref_tutor_train.jsonl` **≥1200 对**；打印长度分布（chosen/rejected 的字符长度均值与 p95）；分差分布里 <2 的已被丢弃。少于 1200 对 → 把 prompt 数从 600 提到 800 重采，不要降低分差门槛。

> 注：强模型打分的 prompt 里**必须要求它先输出评分依据再输出分数**（JSON：`{"reason": "...", "score": 7}`），并且把这条提示词原文存进 `annotation_prompt.md`。理由：DPO 的效果天花板由配对质量决定，面试官一定会问"你的偏好标签哪来的、噪声多大"。**周 8 踩过的坑别再来**：让裁判模型直接给分，它会给出一个 7 就算 7，事后无法追溯。
>
> 注：为什么"规则分 + 模型分"混合而不用纯模型分——纯模型分容易把"更长更啰嗦"当"更好"，DPO 学到的是长度而非质量（周 8 的 reward hacking 观察就是这个）。规则分里 JSON 合法这一项是**硬否决**：不合法的候选绝不能当 chosen。

任务 4：100 条人工复核 + 一致率（约 80 分钟）

1. 从训练对里随机抽 100 对（`random.seed(42)` 抽，把种子写进脚本）。
2. 复核规则（写成 `review_rules.md`，三条）：
   - 遮住分数，只看两个输出，问自己"如果只能给用户看一个，我选哪个"；
   - 若你选的与打分器选的相反 → 记一条 `DISAGREE`，并写一句原因（分类：格式 / 事实 / 相关性 / 啰嗦 / 中文表达）；
   - 若两边都明显差 → 记 `BOTH_BAD`，该对从训练集剔除。
3. 逐条填表 `manual_review_100.csv`：`idx, agree, reason`。

完成判据：**一致率 ≥80%**（`agree=1` 的比例）且 `BOTH_BAD` ≤10 条。一致率 70-80% → 改打分提示词重跑（最多一次）；**<70% → 停下，把数据问题写进 Day 6 博客素材**，并降门槛只用规则分（不合法即 rejected）+ 分差 ≥4 的极端对，宁少勿脏。

任务 5：收工笔记（约 30 分钟）

`NOTES.md` 写 ≥6 条，至少覆盖：本次移植改了哪三处、template 体检结果、配对丢弃率、一致率与 disagreement 主要类别。

完成判据：`NOTES.md` 今天新增 ≥6 条，其中"配对丢弃率"与"一致率"两条必须是**具体数字**（不是"丢弃了一些"）。

今日验收：`PIPELINE SMOKE OK` + `pref_tutor_train.jsonl` ≥1200 对 + 一致率有数（≥80% 或已触发降级）+ NOTES ≥6 条。

Day 2 • 周二：DPO 上卡（tutor 数据）

今日总览（6.5h，其中卡时 4h）：今天要拿到 `dpo-merged` 权重：周 10 的 SFT 模型在 tutor 偏好对上做完 DPO，held-out 偏好对准确率达标，并且**没有把 JSON 格式训崩**。开卡前先把账算清（任务 1），训练期间不许离开终端超过 20 分钟。

任务 1：开卡前算账 + 环境（约 40 分钟）

1. 算账（写进 `m4-capstone/align/COST.md`，开卡前必须落笔）：

项目	估算	依据
卡型	4090 24G 单卡	1.5B + LoRA + bf16，DPO 两模型同卡放得下
本次预算	4h × ¥1.5-2.5 ≈ ¥6-10	AutoDL 按量价
显存粗估	权重 bf16 ≈3GB + ref（关 adapter 复用）≈0 + 梯度 3GB + AdamW 态（只算 LoRA，很小）+ 激活（seq 2048、batch 2、gradient checkpointing）≈6-9GB	预期量级，非实测；以 `nvidia-smi` 为准
排队风险	4090 常无货	备选 4090D / 3090（3090 显存同为 24G，速度约慢三成）

2. 开机、挂上持久盘路径、装环境（**与 Day 1 同一份** `requirements-w11.txt`）：

```
source /etc/network_turbo || true
export HF_ENDPOINT=https://hf-mirror.com
pip install -r /root/autodl-tmp/m4/requirements-w11.txt -q
python -c "import torch,trl;print(torch.__version__, trl.__version__, torch.cuda.is_available())"
```

3. 确认 `torch.cuda.is_available()` 为 `True`，且周 10 merged 权重、`pref_tutor_*.jsonl` 都在 `/root/autodl-tmp` 下。

完成判据：`COST.md` 里今天这一行的预算数字写定（**没写不许开卡**）；环境版本号已记进文件；两份 jsonl 在卡上行数与本地一致（`wc -l`）。

> 注：本地 Windows 侧如果这周要在 CPU 上跑 0.5B 冒烟而 `import torch` 报 `OMP: Error #15 Incident detected`（conda 的 MKL 与 torch 自带 OpenMP 打架），正解是用 Week1 建的那个独立 venv，不要用 conda base；急着往下走就先 `set KMP_DUPLICATE_LIB_OK=TRUE`（cmd）/ `export KMP_DUPLICATE_LIB_OK=TRUE`（Git Bash）绕过，卡上（Linux）不会有这个问题。

任务 2：DPO 训练脚本（约 60 分钟）

`m4-capstone/align/run_dpo_tutor.py` 的关键设置（在你现有代码上核对，缺哪补哪）：

```
from peft import LoraConfig
from transformers import AutoModelForCausalLM, AutoTokenizer
from datasets import load_dataset
from trl import DPOConfig, DPOTrainer

P = "/root/autodl-tmp/m4/sft-merged"
tok = AutoTokenizer.from_pretrained(P)
model = AutoModelForCausalLM.from_pretrained(P, torch_dtype="bfloat16")
ds = load_dataset("json", data_files={"train": "pref_tutor_train.jsonl",
                                      "test": "pref_tutor_test.jsonl"})
peft = LoraConfig(r=16, lora_alpha=32, lora_dropout=0.05,
                  target_modules=["q_proj", "k_proj", "v_proj", "o_proj",
                                  "gate_proj", "up_proj", "down_proj"])
args = DPOConfig(
    output_dir="/root/autodl-tmp/m4/dpo",
    beta=0.1, learning_rate=5e-6,            # DPO 的 lr 比 SFT 低一个量级
    per_device_train_batch_size=2, gradient_accumulation_steps=8,
    num_train_epochs=1.0,
    max_length=2048, max_prompt_length=512,
    bf16=True, gradient_checkpointing=True,
    logging_steps=5, eval_strategy="steps", eval_steps=50, save_steps=100,
    seed=42, data_seed=42, report_to="wandb")
tr = DPOTrainer(model=model, ref_model=None, args=args, peft_config=peft,
                train_dataset=ds["train"], eval_dataset=ds["test"], tokenizer=tok)
tr.train()
tr.save_model("/root/autodl-tmp/m4/dpo-adapter")
```

完成判据：`python run_dpo_tutor.py --dry_run`（自己加 20 行 `--dry_run` 分支：只走 1 个 step 并打印 batch 内一条样本被 tokenize 后的 label，prompt 段必须是 `-100`）打印 `DRY OK` + label 前段全 `-100`。

> 注：`ref_model=None` **只在走 LoRA 时成立**——TRL 会用"关掉 adapter 的同一份权重"当参考模型，所以显存不翻倍。如果你哪天改成全参 DPO，就必须显式传 `ref_model`（同一路径再 load 一份），显存 ×2，4090 上 1.5B 全参 DPO 会紧到不可用。这一条是面试常问的"你 DPO 的 ref 哪来的"，答对能省半分钟。
>
> 注：DPO 的样本里没有 `label=-100` 这种 mask，它是靠 `prompt / chosen / rejected` 三段分别算 logprob 再作差。**但 prompt 段的截断方式仍然重要**：`max_prompt_length=512` 太短会把 tutor 指令截掉一半，chosen/rejected 的 logprob 就建立在不同条件上，loss 会假降。Day 2 的 dry_run 就是查这个。

任务 3：跑训练 + 盯三件事（约 170 分钟）

1. `bash start_w11.sh run_dpo_tutor.py`，wandb 项目名 `m4-align`，run 名 `dpo-tutor-1.5b`。
2. 每 15 分钟看一次，只盯三件事，写进 `train_log.md`：
   - `loss`：应从 ≈0.69（ln2 的起点，说明模型分不清 chosen/rejected）往下走；
   - `rewards/chosen`、`rewards/rejected` 与二者差值（margin）：margin 上行是健康的核心信号；
   - `epoch` 与剩余时间：`预期量级` 是 1200 对 / 等效 batch 16 ≈ 75 step，1.5B + seq2048 在 4090 上**约 1.2-2.2h**（含 eval），显著超 3h 就是哪里卡住了。

完成判据：训练跑完（不是中途停）；wandb 里 `loss` 曲线整体下行、margin 曲线整体上行；`rewards/accuracies`（held-out）最终 **≥0.70**（预期量级 0.70-0.82；若你的周 8 基线本来就在这个区间，以不劣化超过 3 个点为合格线）。

> 注：**"loss 不降"的排查顺序写死**（这是总计划 M1 就练过、面试必问的那条链）：lr 是不是被覆盖成 0（`args.learning_rate` 与 scheduler 冲突）→ 数值（bf16 换 fp32 跑 5 step 看是否还 NaN，即精度问题）→ 数据管线（打印一条样本的 chosen/rejected 是不是拼反了——**DPO 标签反了会让 loss 上涨，这个错出现率极高，先查它**）→ 初始化（adapter 是否真的挂上了，`model.print_trainable_parameters()` 可训参数应 <1%）。**单项定位限时 40 分钟**，超时就带日志回本地看。

任务 4：中途格式诊断（约 40 分钟）

1. 用中间 checkpoint（`steps=50` 与训练结束各一次）跑 20 条固定 prompt（就是 A/B 协议里 30 个场景的前 20 个），统计 JSON 合法率。
2. 若 `steps=50` 时合法率比 SFT 基线**下降 >10 个百分点** → 立即停，按顺序处理：lr 降到 2e-6 → 把偏好对里的长输出对（chosen+rejected 合计 >3000 字符）先剔掉重训 → 仍崩则改用 **只训 0.5 epoch**（早停也是合法方案，写进报告）。

完成判据：`steps=50` 与 final 两个点的 JSON 合法率都记进表；final 合法率 **≥ SFT 基线 -3 个百分点**（SFT 基线今天先测，作为对照）。达不到就必须给出上面三选一的处置并记录结果，不许"先跑完再说"。

任务 5：merge + 本地快检（约 60 分钟）

```
python -m peft_utils.merge_and_save --base /root/autodl-tmp/m4/sft-merged \
       --adapter /root/autodl-tmp/m4/dpo-adapter \
       --out /root/autodl-tmp/m4/dpo-merged
```

（`merge_and_save` 就是周 5/10 用过的那个 30 行脚本：`AutoPeftModel.from_pretrained` → `merge_and_unload()` → `save_pretrained()` + 把 base 目录里的 `tokenizer*` 与 `chat_template.jinja` **一起 copy 过去**。）

拷回本地一份 config/tokenizer/最小样例（权重留在卡上），并在卡上做 20 题人眼看（只判断两件事：JSON 合法、有没有把中文说成人话）。

完成判据：`dpo-merged` 目录里 tokenizer 相关文件齐（`ls` 数 ≥6 个文件）、`smoke_template.py` 指向它仍打印 `HAS_CHATML_MARKERS: True` + `PARSE_OK`；20 题里 JSON 合法 ≥16。

任务 6：关机 + 记账（约 20 分钟）

`git push` 脚本与 jsonl，控制台点**关机**并确认"已关机"，在 `COST.md` 补真实卡时与花费。

完成判据：`git status` 干净且远端有今天的 commit；AutoDL 控制台该实例状态显示"已关机"（**截图或口头确认不算，回控制台看**）；`COST.md` 里 Day 2 行的真实小时数 ≤4h。

今日验收：训练跑完 + held-out acc ≥0.70 + JSON 合法率不劣于 SFT 基线 -3pt + `dpo-merged` 冒烟通过 + COST 有真实数。

Day 3 • 周三：可验证奖励三件套（纯本地，卡时 0）

今日总览（6h）：今天决定 Day 4 的 GRPO 是"有谱"还是"赌博"。把三件事写成可测的代码：**JSON 格式合法**、**CEFR 难度一致**、**释义事实性可核验部分**，并对 200 条真实输出做一次**奖励分布体检**——这一步就是防 reward 不涨的保险。全部本地 CPU。

任务 1：把输出契约写成 JSON Schema + 词典缓存（约 30 分钟）

1. `m4-capstone/align/tutor_content.schema.json`，严格对齐 `TutorContentSchema`：四个键必给、`sceneExamples` `"minItems": 3, "maxItems": 3`、每条含 `en` / `zh` / `scene` 且为非空字符串、`additionalProperties: false`。
2. 建词典缓存 `dict_cache.csv`（本地一次性生成，训练时只读它，**不要在每条 rollout 里打 1GB 的 sqlite**）：

```
# build_dict_cache.py —— 一次性，CPU 约 2-4 分钟
import sqlite3, csv
src = sqlite3.connect("E:/CC/tomorrow-memory/stardict.db")   # 明日记忆仓库内的 ECDICT sqlite 发行
rows = src.execute("select word, frq, collins, oxford, tag, exchange from stardict "
                   "where frq is not null and frq > 0")
with open("dict_cache.csv", "w", newline="", encoding="utf-8") as f:
    w = csv.writer(f); w.writerow(["word", "frq", "collins", "oxford", "tag", "exchange"])
    for r in rows:
        w.writerow([r[0].strip().lower(), *r[1:]])
print("DICT CACHE OK")
```

完成判据：`python -c "import json;print(len(json.load(open('E:/study/posttrain/week11/day3/tutor_content.schema.json'))['required']))"` 输出 `4`；`dict_cache.csv` 存在且行数 ≥50 万；`python -c` 抽 3 个词查得到 `frq`。

> 注：`stardict.db` 的中文 `translation` 字段有编码问题（取出是乱码且不可无损还原），**只有英英 `definition` 与 `frq/collins/tag/exchange` 可靠**。所以事实性奖励只能建立在"词条存在性 + 屈折形式 + 字母重合"这类可机器核验的锚上，中文释义是否准确**只能抽检**，不能进 reward——这条边界今天定死，Day 3 任务 4、Day 5 任务 3 都靠它。

任务 2：三个奖励函数 + 单测（约 90 分钟）

`m4-capstone/align/rewards.py`，每个函数收一个字符串（模型原始输出），返回 `[0,1]` 的浮点：

```
import json, re, csv, functools
from jsonschema import validate, ValidationError

SCHEMA = json.load(open("tutor_content.schema.json", encoding="utf-8"))

@functools.lru_cache(maxsize=1)
def dict_rows():
    out = {}
    with open("dict_cache.csv", encoding="utf-8") as f:
        for row in csv.DictReader(f):
            out[row["word"]] = row
    return out

def parse_obj(text):
    """剥掉 ```json 围栏与前后废话，只取第一个完整对象；失败返回 None"""
    m = re.search(r"\{.*\}", text, re.S)
    if not m:
        return None
    try:
        obj = json.loads(m.group(0))
    except Exception:
        return None
    return obj

def format_reward(text):
    obj = parse_obj(text)
    if obj is None:
        return 0.0
    try:
        validate(obj, SCHEMA); return 1.0
    except ValidationError:
        return 0.0

CEFR_BAND = {"A1": (0, 2000), "A2": (2000, 5000), "B1": (5000, 12000),
             "B2": (12000, 25000), "C1": (25000, 10**9)}
ORDER = ["A1", "A2", "B1", "B2", "C1"]

def cefr_reward(text, target_level):
    """输出用词难度是否与目标档一致：取英文词频 75 分位落在哪一档，与目标比距离"""
    obj = parse_obj(text)
    if obj is None:
        return 0.0
    en = " ".join(e.get("en", "") for e in obj.get("sceneExamples", []))
    words = re.findall(r"[A-Za-z]+", en.lower())
    frqs = [int(dict_rows()[w]["frq"]) for w in words if w in dict_rows()]
    if len(frqs) < 20:
        return 0.0                          # 可判定的词太少，不给分（防靠少写例句拿高分）
    frqs.sort()
    p75 = frqs[int(len(frqs) * 0.75)]
    band = next((lv for lv in ORDER if CEFR_BAND[lv][0] <= p75 < CEFR_BAND[lv][1]), "C1")
    d = abs(ORDER.index(band) - ORDER.index(target_level))
    return {0: 1.0, 1: 0.5}.get(d, 0.0)

def fact_reward(text, word):
    """只吃可机器核验的三条锚，各 1/3"""
    obj = parse_obj(text)
```

```
    if obj is None:
        return 0.0
    ex = dict_rows().get(word.lower(), {}).get("exchange", "")
    forms = {word.lower()} | {p.split(":")[-1] for p in ex.split("/") if ":" in p}
    s1 = sum(1 for e in obj.get("sceneExamples", [])
             if any(re.search(rf"\b{re.escape(f)}\b", e.get("en", "").lower()) for f in forms)) >= 2
    cmp_words = re.findall(r"[A-Za-z]{3,}", obj.get("confusable", ""))
    s2 = bool(cmp_words) and all(w.lower() in dict_rows() for w in cmp_words[:3])
    pieces = re.findall(r"[A-Za-z]{2,}", obj.get("wordRoot", ""))
    src = sorted(word.lower()); got = sorted("".join(pieces).lower())
    s3 = bool(pieces) and sum(1 for c in got if c in src) / max(len(got), 1) >= 0.7
    return (s1 + s2 + s3) / 3.0
```

写 `test_rewards.py`：**15 个用例**，含 4 个正例、6 个负例（缺字段 / `sceneExamples` 只有 2 条 / 多一个 `note` 字段被 `additionalProperties` 挡掉 / 围栏包裹 / 截断 / 例句里换成了别的词）、5 个边界（空串、只有 `{}`、全是中文、词表命中率 <20 词、confusable 提到一个不存在的词）。

完成判据：`cd E:/study/posttrain/week11/day3 && python -m pytest -q test_rewards.py` **15 passed**；三个函数对任一输入的返回值恒在 `[0,1]`（在测试里断言）。

> 注：`fact_reward` 的第三条（词根字母重合 ≥0.7）是个**弱锚**：它抓得住"凭空编一个不存在的拉丁词根"，抓不住"拆得对但解释错"。别把它当事实性检验的全部，写注释说明它的边界。面试官问"你的可验证奖励怎么定义事实性"，标准答案是：**能自动核验的进 reward，需要世界知识的进评测与人工抽检**——这就是 RLVR 里那个 V 为什么必须是 verifiable。

任务 3：奖励分布体检（约 90 分钟，今天最重要的一件事）

1. 用 **Day 2 的** `dpo-merged` 对 200 条 prompt（`temperature=1.0, seed=42`）生成输出——这步要卡时 0.5h，或复用 Day 2 采样缓存。
2. 本地跑三个奖励，出表 `reward_calibration.md`：每个奖励的**均值 / 中位数 / 标准差 / 组内（同 prompt 4 个输出）极差均值**。

完成判据：三个奖励的均值都落在 **0.15-0.85** 区间；且同 prompt 的组内极差均值 **>0.05**。任一奖励均值 <0.15 或 >0.85、或组内极差 ≈0 → **必须在 Day 4 之前改掉它**（放宽/收紧阈值或换权重），否则 GRPO 的组内优势 `(r_i - mean)/std` 会退化：std≈0 时优势全为 0，梯度为 0，reward 曲线就是条直线，跑 6 小时等于没跑。

> 注：这一张表就是 Day 4 的"能不能开跑"的门禁，也是博客第 2 篇最值钱的一段——`reward 不涨` 的原因十个里有六个能在采样阶段就看出来，根本用不着等训练。把这张表拍照进 repo。

任务 4：组合奖励 + 硬否决（约 90 分钟）

```
def combined(text, word, target_level):
    f = format_reward(text)
    if f == 0.0:
        return -0.5                          # 格式不合法：硬否决，直接负分
    c = cefr_reward(text, target_level)
    a = fact_reward(text, word)
    r = 0.35 * c + 0.35 * f + 0.30 * a
    n = len(text)
    if n > 900:                              # 长度软惩罚，防"写更多分"的 hacking
        r -= min(0.3, (n - 900) / 1500)
    return max(-1.0, min(1.0, r))
```

完成判据：6 个手工构造的极端样例（合法满分式 / 只有 JSON 没内容 / 啰嗦 1200 字但全合法 / 编造 confusable / 例句全用同一个词 / 输出纯中文）的 `combined` 值**逐一打印并与你的直觉预期一致**，把"预期 vs 实际"写进 `reward_design.md`，任何一条反直觉都要解释清楚为什么。

> 注：`-0.5` 这个硬否决是设计如此：如果格式不合法只给 0 分，而某个"合法但内容平庸"的输出给 0.2，那模型学到的仍然是格式；但当不合法给 -0.5 且**占比 >50%** 时，组内优势会被格式项吃满，内容学不动——所以 Day 3 任务 3 里"格式奖励均值 ≤0.85"这个判据要和这里一起看。面试问"你怎么平衡硬约束和软质量"，这就是答案的一半。

任务 5：接进 GRPO 并本地空跑（约 60 分钟）

1. 准备 `grpo_prompts.jsonl`：字段 `{prompt, word, target_level}`（**只有 prompt，没有答案**——RLVR 不需要标签，这是它和 SFT 最本质的区别）。用 300 条（800 条留 Day 4 正式用）。
2. 本地 0.5B 空跑 2 step：

```
python run_grpo_tutor.py --model Qwen/Qwen2.5-0.5B --prompts grpo_prompts.jsonl \
  --max_steps 2 --num_generations 4 --beta 0.04 --learning_rate 1e-6 \
  --temperature 0.9 --max_completion_length 512 --report_to none
```

完成判据：打印 `GRPO LOCAL OK`；两个 step 的 `loss` 均非 NaN；`rewards/combined/mean` 有打印且落在 [-0.5, 1.0]；`kl`（或 `loss/kl`）有值。

今日验收：15 单测全绿 + `reward_calibration.md` 三门指标齐 + 6 条极端样例核对齐 + `GRPO LOCAL OK`。**卡时今天 0**（Day 3 任务 3 的采样若需要卡，记 0.5h 进 COST）。

Day 4 • 周四：GRPO 上卡（本周最大风险）

今日总览（7h，卡时 6h）：把 Day 3 的可验证奖励接到 `dpo-merged` 上跑 GRPO。今天的目标**不是"必须涨"**，是"必须给出可辩护的结论"：要么 reward 上行且格式没崩，要么按降级方案退回 DPO-only 并写清定位过程。

任务 1：开卡前算账 + 显存/时长估算（约 40 分钟）

1. 填 `COST.md` 的 Day 4 行：6h × ¥1.5-2.5 = **¥9-15**（4090）。**若 Day 3 的分布体检没过 → 今天不许开卡**，先修奖励，卡时留到 Day 5 用。
2. 估算表（写进 `grpo_plan.md`）：

量	取值	说明
prompt 数	300（跑 2 个 epoch）	400 更稳但时间不够，按 300 起
G（num_generations）	4	8 的组内基线更准但生成量翻倍，先 4
每 step 生成条数	4 prompt × G 4 = 16	等效 batch = per_device_batch × 累积步数
max_completion_length	512	按 Day 1 缓存样本的输出长度分布定（**预期量级 300-450 token**，截断比例要在任务 2 的监控里看）
rollout 引擎	先用内置 generate（**不开 vLLM**）	稳，慢；跑通再谈加速
单 step 时长	预期量级 30-60s	1.5B、16 条生成、4090
总时长	300×2/4 = 150 step ≈ 1.5-2.5h	剩 3h 给排队、重跑、诊断

完成判据：`COST.md` 与 `grpo_plan.md` 两张表落笔，且口算得出不超过 6h 的总时长；写不出来就不要开卡。

任务 2：跑 GRPO + 三曲线监控（约 240 分钟）

1. `bash start_w11.sh run_grpo_tutor.py`，wandb run 名 `grpo-tutor-1.5b-g4`。
2. **必须同时记录三条曲线**（缺一即判据不满足）：`rewards/combined/mean`（上行？）、`kl`（是否受控）、`completions/clipped_ratio` 与平均输出长度（是否暴涨）。
3. 每 20 分钟把当前值抄进 `monitor.md`（一行一次，含时间戳）。抄表的目的不是记录，是逼你每 20 分钟做一次判断——GRPO 的翻车大多在前 30 step 就有征兆。
4. 每 50 step 用缓存的 20 题固定 prompt 测一次 JSON 合法率（脚本 `check_format.py`，本地 30 秒）。

完成判据（收敛口径，三条全满足才算"收敛"）： - `rewards/combined/mean` 从起点到终点**上行 ≥15%（相对）**，或达到 0.7 以上（取先满足者）； - `kl` 全程 **<10**（预期量级 0.5-8；不同实现口径有差异，以你自己的起点为参照，**只要单调爆涨就判失败**）； - 平均输出长度增幅 **<40%**，且 JSON 合法率不低于 Day 2 的 DPO 基线 -3pt。

> 注：**reward 不涨的排查顺序（照做，别跳）**：① 组内 std 是否 ≈0（Day 3 体检已排除，但换模型后要重看：`dpo-merged` 可能比 0.5B 更"整齐"，4 个输出几乎一样 → 把 `temperature` 从 0.9 提到 1.0-1.1，或 `top_p=1.0`）；② 奖励函数在真实输出上是不是恒为常数（`rewards/*/mean` 打印出来看）；③ lr 是否太小（1e-6 是稳妥起点，跑 100 step 毫无动静可试 2e-6，一次只改一个）；④ KL 系数 `beta` 太大把模型钉死在 ref 上（0.04 → 0.01）；⑤ 数据太难（全部输出格式都不合法，模型没能力从"改进一点点"里获益）。**每一步都要留一条 wandb 截图或一行日志**——这些就是 Day 6 博客与周 14 失败案例清单的原材料。
>
> 注：**输出长度暴涨而 reward 平 = reward hacking**，这是教科书信号，出现就立刻停（不要"再等等看"）。停完先检查长度惩罚系数与 `max_completion_length` 截断比例，再决定重跑。这一条如果发生，你今天反而赚了：亲手抓到一次 hacking

> 比读十遍定义有用。

任务 3：中途降级判定（约 60 分钟）

跑满 **80 step** 后做一次**书面**判定（写进 `grpo_verdict.md`，三选一）：

判定	条件	动作
A 继续	reward 上行 ≥8%（80 step 内）且格式没崩	跑到 2 epoch 结束
B 改一处重跑	reward 平的但组内 std >0.05（说明有信号没被利用）	按上面排查顺序取**一条**改，重跑 80 step 再判
C 降级	组内 std ≈0、或格式必崩、或 160 step 后 reward 仍无方向	**退回 DPO-only**，今天收工

完成判据：`grpo_verdict.md` 里有明确的一个字母 + 支撑它的两条数字。**C 路线完全合规**：总计划风险 1 写明"GRPO 不收敛时用 1.5B + 小数据把 pipeline 跑通、记录失败曲线同样是面试素材"。但走 C 必须在文件里写清三行：现象是什么、你定位到哪一步排除了什么、为什么最终选 DPO-only（这三行周 12 的博客第 3 篇会直接引用）。

任务 4：保存与备份（约 60 分钟）

1. 保存最后一个 checkpoint 与 `steps=100` 的中间点（各存 adapter 即可，别存全参）。
2. merge 成 `grpo-merged`（同 Day 2 任务 5 的脚本），跑 `smoke_template.py` 指向它。
3. 把三条曲线的 wandb 截图存 `m4-capstone/align/curves/`（连不上 wandb 就用 `WANDB_MODE=offline` 的本地目录 + 自己 `pandas` 画图，别为截图反复折腾网络）。

完成判据：`grpo-merged`（或降级后的说明）+ 至少 3 张曲线图入库 + 图里每条线的含义能在 30 秒内讲清。

任务 5：关机 + 记账（约 20 分钟）

控制台确认"已关机"，`COST.md` 填真实卡时；若今天用到 6h 上限，Day 5 的生成任务改本地 0.5B 或顺延 1.5h（本周 14h 总额内挪）。

完成判据：控制台"已关机"；`COST.md` 的累计卡时 ≤10.5h（Day 1 的 0.5h + Day 2 的 4h + 今天的 6h）。

今日验收：三曲线齐 + `grpo_verdict.md` 判定明确 + merge 冒烟通过 + COST 更新。

Day 5 • 周五：对齐前后三档对比表 + 人工盲评

今日总览（6h，卡时 2h）：今天把 M4 对齐阶段变成一张能贴进简历的表。三档模型（`sft-merged` / `dpo-merged` / `grpo-merged`，降级则两档）在同一批题上跑，指标全部来自 Day 3 的奖励函数与今天的盲评，**不许出现没有出处的数字**。

任务 1：三档生成（约 90 分钟）

1. 出题：从 800 条 prompt 里 `random.seed(42)` 抽 **100 条 held-out**（必须与 Day 1 配对用的 600 条**不重叠**，用 13-gram 交集自查一遍重叠为 0）。
2. 三个模型各跑 100 条，**采样参数完全一致**（`temperature=0.2, top_p=0.9, seed=42, max_new_tokens=512`）。推理服务就用卡上的 `python -i` 循环，2h 卡时够用（300 次生成，1.5B，预期量级每次 2-4s）。
3. 落盘 `gen_sft.jsonl` / `gen_dpo.jsonl` / `gen_grpo.jsonl`。

完成判据：3×100 = 300 条输出齐、无缺失；每条记录带 `model, word, target_level, text, n_chars`。

> 注：温度写死 0.2 的理由要能讲：评测要的是**可复现的能力读数**，不是采样多样性；GRPO 训练里 `temperature` 用 0.9-1.0 是为了探索。面试问"你评测用什么解码"，答"贪心/低温 + 固定 seed，并对边界样本另做 3 次重跑看稳定性"。

任务 2：自动指标表（约 90 分钟）

`metrics.py` 对每档算五项：JSON 合法率、CEFR 一致率（`cefr_reward` 取 ≥0.5 记命中）、事实锚命中率（`fact_reward` ≥0.67）、平均输出字符数与 p95、在 `pref_tutor_test.jsonl` 上的偏好对准确率（chosen logprob > rejected logprob 的比例，`sft-merged` 也照算，作对照）。

完成判据：`RESULTS.md` 里有一张 4 列（模型 × 指标）主表 + 一行结论；数字**全部来自脚本输出**（脚本文件路径写在表下方）。

任务 3：40 题人工盲评（约 90 分钟）

1. 盲评规则（写进 `blind_rules.md`）：
   - 抽 40 条，两两成对比较 `dpo` vs `grpo`（降级则 `sft` vs `dpo`）；
   - 比较前**删除所有长度线索**（脚本按字符数截断到两者较短值再展示），顺序用 `random.seed(7)` 随机 A/B；
   - 三档判定：A 好 / 差不多 / B 好；`差不多` 算平，不拆成 0.5 分；
   - 每判"好"必须写一句理由（口径就用 Day 1 的四类：格式 / 事实 / 相关性 / 中文表达）。
2. 填 `blind_40.csv`。

完成判据：40 条全判完、每条有理由；**胜出方胜率 ≥55%**（预期量级：DPO 对 SFT 通常 55-70%，GRPO 对 DPO 可能只有 45-60%）。**低于 50% 不算失败**，照实写——盲评输给对手但自动指标涨，几乎一定说明奖励设计和真实质量脱钩了，这是最有价值的发现，写进 Day 6 博客。

任务 4：能力退化快测（约 60 分钟，可砍项）

30 题小测（GSM8K 里抽 20 题 final-answer 数值比对 + 10 道中文常识问答，题库存 `m4-capstone/align/degen30.jsonl`），跑 **原始基座** `Qwen2.5-1.5B` 与三档对齐模型。

完成判据：四行一列的表，且**任一档相对基座退化 ≤5 个百分点**；超过就把"周 12 必须做正式通用基准并定位退化来源"写进 gaps 第一条。

> 注：这 30 题只是烟雾报警器，**不能当周 12 的替代**：题量太小，单题就是 3.3 个点，噪声盖过信号。周 12 会用 `lm-eval-harness` / OpenCompass 在正式基准上带 base 对照重做，今天的数字只用来决定"要不要提前准备 replay 混数据"。

任务 5：写进 RESULTS.md 并 push（约 30 分钟）

表 + 三曲线图引用 + 一句 200 字内的本周结论（说清"哪一档是最终交付、为什么"）。

完成判据：`git push` 成功；`RESULTS.md` 里每个数字旁边都有脚本或文件路径；结论段 ≤200 字。

今日验收：三档生成齐 + 自动指标主表 + 盲评 40 条有理由 + 退化快测表（或已按砍单处理并记 gaps）+ `RESULTS.md` 已 push。

Day 6 • 周六：M4 系列博客第 2 篇 + 失败案例归档 + 周复盘

今日总览（6h，卡时 ≤0.5h）：今天把本周的翻车与救场变成公开输出。博客第 2 篇是 M4 系列里最该写技术细节的一篇——它的价值不在"我做完了"，在"我知道它为什么不动"。

任务 1：博客成文（约 120 分钟）

1. 标题（三选一或自拟）：《明日记忆 Tutor 对齐实战：reward 不动的那六小时》《DPO 之后，我为什么差点把 GRPO 删掉》《可验证奖励不是写个 if 就行》。
2. 结构（**1800-2500 字**，M4 系列第 2 篇，博客配额到此为止：周 5、周 7、周 9、周 10-12 三篇，共 6 篇，不要多写）：
   - ① 问题设定：为什么 tutor 任务适合 RLVR（有可验证锚），哪里不适合（中文释义准确度）；
   - ② 三个奖励函数的定义与**它们的失效边界**（贴 Day 3 那段最关键的代码 + 分布体检表）；
   - ③ 一条真实失败曲线：现象 → 你怀疑的三种解释 → 逐个排除的证据 → 结论（**贴原始报错或日志行，不要转述**）；
   - ④ 对齐前后对比表（截 Day 5 的 `RESULTS.md`）；
   - ⑤ 差异声明：哪些是自己写的、哪些来自 TRL / 周 8 / 周 9 的既有脚本（README 同款诚实标注）。

完成判据：草稿 ≥1800 字、图 ≥3 张（曲线 / 分布直方图 / 对比表截图）、**至少 3 段真实报错或日志原文**。

任务 2：发布（约 60 分钟）

发到《从全栈工程师到后训练》专栏，检查代码块在目标平台的渲染，链接贴到 `NOTES.md` 顶部与 `m4-capstone/README.md`。

完成判据：链接可匿名访问（无痕窗口打开一次）。

任务 3：失败案例归档（约 80 分钟）

`m4-capstone/align/FAILURES.md`，本周所有翻车逐条写成**四段式**：`现象（含原始报错行）→ 我怀疑什么 → 用什么证据排除 → 最终修复/绕过`。

完成判据：**≥6 条**，其中至少 2 条来自今天之前（Day 2 的中途诊断、Day 4 的降级判定）。这一份是**周 14「失败案例复盘」的主素材**，本周不写出来，两周后你就再也想不起当时的报错原文了。

任务 4：周复盘 review.md + gaps 清账（约 70 分钟）

五块：① 硬产出清单（带路径/链接）；② 卡时与花费真实值 vs 14h/¥28 预算（对照 `COST.md`）；③ 本周最贵的一次判断失误；④ 现在能闭卷讲清的三件事；⑤ 留给周 12 的三个洞（必须是具体可验证的表述，不写"评测要加强"这种）。

完成判据：五块齐、gaps 每条有 `[x]` 或"哪天+动作"。

任务 5：缓冲（约 30 分钟）

前面顺就提前收工；不顺则用于补 Day 5 未完成项。

完成判据：要么提前收工（`git log` 里今天最后一个 commit 早于 23:00），要么动用缓冲并把它花在哪个缺口上写进 `gaps.md`（一行即可，不许"随便看看"）。

今日验收：博客链接可访问 + `FAILURES.md` ≥6 条 + `review.md` 五块齐。

Day 7 • 周日：补漏 + 闭卷复测 + 总验收 + 周 12 预习

今日总览（4.5h）：今天不排新训练。上午补漏 + 闭卷复测，中午做总验收，下午 30-40 分钟预习周 12。

任务 1：补漏（约 100 分钟）

打开 `gaps.md` 与 `review.md`。**规则：最多补 2 个洞，贪多必失。**优先级：奖励分布体检没过 → 盲评胜率异常没定位 → 其余。

完成判据：这 2 个洞在 gaps 里从 `[ ]` 变 `[x]`，并各写一句"怎么补的"。

任务 2：闭卷复测（约 45 分钟，别跳）

拿一张纸，不看任何资料，写完三题（写完再对照，错的当天重写一遍）： 1. 写出 DPO 损失（含 π_{ref} 项与 β），并用两句话说清 β 变大/变小各自的后果； 2. 写出 GRPO 的组内优势 $(r_i - mean)/std$ 的来由：为什么它能替掉 PPO 的 value 网络、代价是什么； 3. 不查文件，写出你本周三个奖励函数的名字、返回值区间、以及**格式不合法时组合奖励给多少、为什么这么设计**。

完成判据：三题**全对**（第 3 题要求数值与 Day 3 代码一致）。错则重做一遍，明天早上再看一次；**这就是周 15 项目深挖要考的口径**，现在错比 12 月错便宜得多。

任务 3：总验收（约 45 分钟）

逐项勾选，输出到 `review.md` 末尾：

#	验收项	判据
1	偏好数据	训练对 ≥1200、test 不重叠（13-gram 自查 0 命中）、100 条复核一致率有数
2	DPO	跑完不中断、held-out 偏好对 acc ≥0.70、JSON 合法率不低于 SFT 基线 -3pt
3	可验证奖励	15 个单测全绿 + `reward_calibration.md` 三项门禁（均值 0.15-0.85、组内极差 >0.05）齐
4	GRPO	三曲线齐 + `grpo_verdict.md` 有 A/B/C 判定与支撑数字（选 C 也记通过，前提是三行说明齐）
5	对比表	`RESULTS.md` 三档 × 五指标，每个数字有脚本出处
6	盲评	40 条全判、每条有理由、规则文档齐
7	公开输出	博客第 2 篇链接可访问 + `FAILURES.md` ≥6 条 + `NOTES.md` 本周 ≥20 条

≥6 项通过 = 本周合格；不足 6 项，把缺的写进周 12 Day 1 的前 2 小时（**周 12 的评测依赖本周的 `dpo-merged` / `grpo-merged` 与 100 题 held-out，缺了它下周第一天就没法开工**）。

完成判据：验收表 7 行里 ≥6 行打了勾，且勾是靠**打开文件/跑脚本**确认的（凭印象打勾视为该项未通过）。

任务 4：下周预习（约 35 分钟）

周 12 是 M4 收官：通用基准（`lm-evaluation-harness` 或 OpenCompass，带 base 对照）、自建 200 题教学评测集 + LLM-as-judge、明日记忆真实接入小模型做 A/B（vLLM 起服务）、模型上传 HF/ModelScope、可复现 repo 收口。今晚只做三件事： 1. 本地 `pip show lm-eval`（没装先别装，Day 1 有步骤）； 2. 想清楚一件事并写进 NOTES：**"防止对齐造成通用能力退化"为什么必须有 base 对照**（因为绝对分数没法归因，只有同一份评测脚本下 base 与后训练模型的差值才说明是"对齐掉的"）； 3. 数一下本周的 `FAILURES.md` 有几条会直接变成周 12 评测项——少于 2 条说明你本周的问题没被量化。

完成判据：三件事都有落笔——`NOTES.md` 里有一段"为什么必须 base 对照"的自问自答（≥3 行）、`FAILURES.md` 的计数与映射结论写在同一页、`pip show lm-eval` 有输出结果记录。

任务 5：缓冲与移交（约 45 分钟）

今天剩的时间不要拿去预支周 12 的训练。只做一件事：把本周**没闭环的东西**打包成周 12 Day 1 的开工清单——一条一行，格式"缺什么 → 在哪个文件里能看到现状 → 周 12 哪天补"。典型会有三类：Day 5 被砍掉的退化快测（周 12 Day 1 的正式基准会一并覆盖）、盲评胜率异常但未定位的档位（周 12 Day 2 的 200 题评测集是它的放大版）、`FAILURES.md` 里还没写"修复"那一列的条目（周 12 的 repo 收口要求每条有结论）。

完成判据：`gaps.md` 顶部新增一节 `## 交给周 12`，条目 ≥3 且每条三个字段齐。**写不出"在哪个文件里能看到现状"的条目，说明它还没被量化，本周不许就这样把它推给下周。**

今日验收：验收表 ≥6 项打勾 + 闭卷三题全对（或已重做）+ 预习 3 步齐 + `## 交给周 12` ≥3 条。

三、常见问题与纪律

- **某天没完成怎么办？** 不要熬夜追。按"天-任务号"写进 `gaps.md`，Day 6 任务 5 与 Day 7 就是给它们准备的。**永远优先保 Day 3 的奖励体检和 Day 4 的 GRPO 判定**：这两件是本周的真实能力增量，博客和盲评都是包装。
- **"reward 不涨我是不是该换算法"**：先换奖励函数、再换超参、最后才换算法。顺序反了你什么都不知道——PPO 和 GRPO 在奖励定义错的时候表现一模一样（都不涨）。今天不涨 ≠ 下周也不会，但**你今天留下的日志能决定下周会不会涨**。
- **卡住多久该求助**：单点超过 40 分钟 → 先写 NOTES，再查（`trl` 仓库 issues、官方文档对应版本的 example），20 分钟无解就跳过并立 gaps 条目。GRPO 环境问题（vLLM 版本、依赖冲突）在周 9 已知是最大坑，本周特意**不开 vLLM**，别自己加回来显得能干。
- **GPU 花费**：本周上限 14h / ≈¥28。超了就去看 AutoDL 控制台的计费明细，九成是忘关机。Day 3 全天 0 卡时，如果发现自己在卡上待了半天，那说明你把 Day 4 的活提前干了。
- **数据合规**：本周新增的偏好标签来自强模型 API（花费 ≤¥20，含在周预算内），公开 repo 里**只放合成与去标识后的数据**；明日记忆真实日志的匿名化在周 7 已做，不要在这周图省事把原始日志推上 GitHub。
- **AI 工具的使用边界**：允许——解释报错、对照 TRL 版本差异、review 你写完的奖励函数。不允许——替你写 reward 设计（那是本周的全部意义）、替你写盲评理由（盲评不是走过场，写理由才是产出）。
- **别把 DPO-only 当失败**。如果你走完 B 路线仍判定 C，一句"我试过 GRPO，组内优势塌缩，我定位到是采样方差不足，退回 DPO 并说明代价"——这句话在后训练面试里的分量，高于一次侥幸的 reward 上涨。